TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

Architecture, Design Decisions & Engineering Analysis

1. Introduction

1.1 What is RAG in This System

Retrieval-Augmented Generation (RAG) is an approach that separates knowledge storage from language generation. Rather than relying on the LLM's parametric knowledge (which may be outdated or hallucinated), the system first retrieves relevant passages from a trusted document corpus, then conditions the language model's output strictly on those passages. In this implementation, a customer-uploaded PDF serves as the sole knowledge base — the LLM (Phi-2) is explicitly prompted to refuse answering from outside knowledge.

1.2 Why RAG for Customer Support

- Customer support requires accurate, document-specific answers — general LLM knowledge is insufficient and potentially harmful
- Support documents (manuals, FAQs, policies) change frequently; RAG allows live updates without model fine-tuning
- Grounding checks provide an auditability layer that pure LLM responses lack
- Escalation logic provides a safety net for queries outside the document's scope

2. System Architecture — Deep Dive

2.1 Two-Phase Architecture

The system operates in two distinct phases:

Phase 1 — Offline Indexing (runs once at session start):

- PDF is loaded via PyPDFLoader, producing one Document per page
- Documents are split into 500-char chunks with 50-char overlap
- all-MiniLM-L6-v2 encodes all chunks into 384-d vectors
- ChromaDB indexes all vectors in-memory using HNSW

Phase 2 — Online Serving (runs per query in while loop):

- Phi-2 rewrites the user query for retrieval clarity
- ChromaDB retrieves top-5 candidates via L2 similarity
- CrossEncoder re-ranks candidates to top-3 by relevance
- Intent detection and confidence computation run in parallel (local Python, no I/O)
- Routing logic decides: escalate, fallback, or proceed to generation
- Phi-2 generates answer constrained to retrieved context
- Semantic grounding validates answer against context via cosine similarity
- Response assembled with debug metadata and source citations

2.2 Component Interaction Matrix

Component	Depends On	Depended By
PyPDFLoader	PDF file, pypdf library	RecursiveCharacterTextSplitter
RecursiveCharacterTextSplitter	LangChain, PyPDFLoader output	ChromaDB indexing, HuggingFaceEmbeddings
HuggingFaceEmbeddings (all-MiniLM-L6-v2)	sentence-transformers, PyTorch	ChromaDB, semantic_grounding()
ChromaDB	HuggingFaceEmbeddings, chunks	retrieve()
CrossEncoder (ms-marco-MiniLM-L-6-v2)	sentence-transformers	rerank() inside retrieve()
Phi-2 (AutoModelForCausalLM)	transformers, torch, GPU	rewrite_query(), generate_answer()
chat()	All above components	CLI while loop

3. Design Decisions

3.1 Chunk Size: 500 characters, Overlap: 50

Rationale: 500 characters (~80-100 words) is a pragmatic balance. Too small and individual chunks lack enough context for the LLM to generate coherent answers; too large and retrieval precision suffers (less relevant material dilutes the context). The 50-character overlap (10% of chunk size) preserves sentence boundary context across chunk edges. This setting was not tuned with evaluation metrics in the notebook — it represents a reasonable default for general-purpose document QA.

3.2 Embedding Model: all-MiniLM-L6-v2

Rationale: This model delivers strong semantic similarity performance for its size (22M parameters). It is one of the most widely benchmarked bi-encoder models on the MS MARCO and BEIR retrieval benchmarks. Crucially for Colab deployment, it runs on CPU without significant latency overhead. The 384-d output is compact enough to not create ChromaDB memory pressure even for large documents.

3.3 Two-Stage Retrieval: Bi-encoder + CrossEncoder

Rationale: Bi-encoders (MiniLM) are fast but approximate; they compute query and document embeddings independently. CrossEncoders process query-document pairs jointly, achieving superior relevance scoring at the cost of higher latency. The system uses bi-encoder for recall (top-5) and CrossEncoder for precision (re-rank to top-3). This is the industry-standard approach used in production dense retrieval systems (e.g., DPR + CE pipelines).

3.4 LLM Choice: Microsoft Phi-2 (2.7B)

Rationale: Phi-2 is a small language model with demonstrated strong performance on reasoning tasks relative to its size. At float16, it fits in a Colab T4 GPU's 16GB VRAM with room to spare. Temperature=0.3 was chosen to minimize hallucination while retaining slight output variability. The 150 token output cap prevents verbose, off-topic generation.

3.5 Prompt Engineering Approach

The build_prompt() function implements a strict constraint prompt with explicit rules:

- Answer ONLY using the given context — enforces grounding
- If answer is NOT clearly present → say 'I don't know' — provides fallback language
- Do NOT use outside knowledge — prevents Phi-2 parametric knowledge leakage
- Be concise and structured — shapes output format

This is a rule-based system prompt, not a few-shot prompt. No examples are provided to the model.

3.6 Grounding Verification: Semantic Cosine Similarity

semantic_grounding() computes cosine similarity between the answer embedding and the first 1000 characters of the context embedding. The threshold of 0.55 was not derived from an evaluation dataset

— it is a heuristic value. The dead-code function is_answer_grounded() (word overlap method) suggests the developer explored multiple grounding strategies.

3.7 Confidence Metric: Mean L2 Distance

compute_confidence() uses mean L2 distance from ChromaDB as a confidence proxy. This is an unconventional choice — L2 distances are not normalized scores and their absolute values depend on the corpus size and embedding space. A more robust approach would use normalized cosine similarity or the CrossEncoder scores. The threshold of 0.4 is a fixed heuristic.

4. Workflow Explanation

4.1 LangGraph Clarification

langgraph==0.0.32 is installed but not used for graph construction. The system's workflow is the chat() function — a linear Python function with conditional branching. This means there is no formal graph state, no parallelism, no interrupt/resume capability, and no graph visualization that LangGraph provides. The design is equivalent to a simple sequential pipeline with exit conditions.

4.2 Node-by-Node Walkthrough

Step	Code	Key Logic
1. Query Rewrite	rewrite_query(query)	Phi-2 generates a cleaner version of the query. Wrapped in try/except — any exception falls back to original. No validation that rewrite is better than original.
2. Retrieval	retrieve(query, k=5)	ChromaDB top-5 by L2, then CrossEncoder re-ranks to top-3. Returns docs[] and scores[] (original L2, not CE scores).
3. Intent Detection	detect_intent(query)	Keyword scan of lowercased query. Order: complaint → summary → process → faq → unknown. First match wins.
4. Confidence	compute_confidence(scores)	Mean of L2 scores. Lower L2 = better match, but threshold 0.4 treats it as a direct score — semantically questionable.
5. Context Guard	len(context.strip()) < 50	Hard minimum context length check. Prevents LLM from generating from no information.
6. Generation	build_prompt() + generate_answer()	Formats context + query into structured prompt. Phi-2 generates up to 150 new tokens.
7. Grounding	semantic_grounding(answer, context)	Embeds answer and first 1000 chars of context. Cosine similarity threshold 0.55.
8. Routing	if/elif chain	Evaluates conditions in priority order. First match determines response type.
9. Output	response assembly	Appends intent, confidence, grounded flag, and source snippets (first 120 chars of each).

5. Conditional Logic Analysis

5.1 Intent Detection — Implementation

detect_intent() is a deterministic keyword-based classifier with priority ordering:

Intent	Trigger Keywords	Notes
complaint	refund, complaint, issue, problem, not working	Highest-priority check; triggers immediate HITL escalation
summary	summarize, summary, overview	No special routing — generates normally
process	how, steps, process, procedure	No special routing — generates normally
faq	what, when, where, which	No special routing — generates normally
unknown	(no keyword matched)	Triggers clarification request, no LLM answer

Limitation: The keyword-based approach can produce false positives ('what is the process' matches 'process' before 'what'). First-match-wins means 'how do I get a refund' matches 'complaint' because 'refund' precedes 'how' in the if/elif order.

5.2 Routing Decision Tree

The full routing logic in chat() evaluates conditions in this exact order:

Order	Condition	Before/After LLM	Response
1	not docs OR context < 50 chars	Before LLM	[ESCALATED] No relevant info
2	build_prompt + generate_answer	LLM called here	—
3	semantic_grounding == False	After LLM	Soft fallback (no escalation tag)
4	intent == 'complaint'	After LLM	[ESCALATED] Complaint
5	confidence < 0.4	After LLM	[ESCALATED] Low confidence
6	intent == 'unknown'	After LLM	Clarification request
7	Default	After LLM	Full answer + DEBUG + SOURCES

6. HITL Implementation

6.1 Role of Human Intervention

The HITL mechanism serves as a quality gate — when the system cannot confidently answer from the document, it acknowledges its limitation rather than generating a potentially incorrect answer. The three [ESCALATED] triggers cover distinct failure modes:

- Empty retrieval: The query has no matching content in the document

- Complaint intent: Policy decision to always route complaints to humans regardless of answer availability
- Low confidence: Retrieval scores are poor, suggesting the document may not cover this topic

6.2 Benefits

- Prevents confident-but-wrong answers reaching end users
- Complaint routing ensures sensitive interactions get appropriate human attention
- Grounding check adds a semantic quality layer beyond keyword matching
- DEBUG INFO footer enables human review of why an answer was given or escalated

6.3 Trade-offs and Gaps

Trade-off	Current Behavior	Production Recommendation
LLM called for complaint queries	Phi-2 runs before complaint check	Move intent check before LLM call to save compute
No real handoff mechanism	Returns [ESCALATED] string only	Integrate with ticketing API (Zendesk, Freshdesk)
No escalation logging	No persistent record of escalations	Log to database with query, session ID, timestamp
Hard-coded confidence threshold	0.4 fixed in code	Calibrate on evaluation set; make configurable
Grounding threshold is a heuristic	0.55 cosine similarity	Evaluate on gold-standard QA pairs to calibrate

7. Challenges & Trade-offs

Challenge	Current Trade-off	Engineering Impact
Accuracy vs. Latency	Two Phi-2 calls (rewrite + generate) for accuracy; ~5-10s total latency	Unsuitable for real-time support chat without GPU batching or model quantization
Chunk size vs. Context Quality	500 chars preserves precision; larger chunks would give more context per retrieval	Small chunks may cut answers mid-sentence; overlap partially mitigates this
Cost vs. Performance	Open-source stack (no API costs); but requires GPU hardware	Colab-dependent; not deployable on serverless CPU-only infrastructure
Grounding strictness vs. Answer rate	0.55 cosine threshold; too strict → more fallbacks, too loose → wrong answers pass	Threshold requires calibration against real user queries
L2 score as confidence	L2 distance used as confidence; not semantically calibrated	Could produce false escalations for niche topics with inherently lower similarity
Single PDF limitation	New PDF requires full re-indexing; no multi-document support	Limits real-world applicability to single-document support scenarios
Dead code (is_answer_grounded)	Word-overlap grounding unused; semantic grounding used instead	Technical debt; unused function should be removed or integrated

8. Testing Strategy

8.1 Testing Approach in the Notebook

The notebook does not include a formal test suite or evaluation framework. Testing is performed via the interactive CLI while loop. No evaluation metrics (RAGAS, BLEU, ROUGE, precision@k) are computed.

8.2 Recommended Test Query Categories

Test Category	Example Query	Expected Behavior
Happy path (FAQ)	'What is the return policy?'	Answer generated with SOURCES and DEBUG INFO
Happy path (process)	'How do I submit a warranty claim?'	Step-by-step answer from document
Complaint escalation	'I have a problem with my order'	[ESCALATED] Complaint detected
Out-of-scope query	'What is the capital of France?'	[ESCALATED] No relevant info OR low confidence

Test Category	Example Query	Expected Behavior
Ambiguous query	'Tell me more'	Unknown intent → clarification request
Empty document coverage	Query about topic not in PDF	Empty context → [ESCALATED] or low confidence
Summary request	'Give me a summary of this document'	LLM generates summary from retrieved chunks

8.3 Recommended Production Evaluation

- RAGAS framework: Faithfulness, Answer Relevance, Context Precision, Context Recall
- Retrieval evaluation: Precision@3 and Recall@3 against a manually curated QA test set
- Grounding threshold calibration: ROC curve on (answer, context) pairs with human labels
- Confidence threshold calibration: Calibration plot of L2 score vs. answer correctness

9. Future Enhancements

Enhancement	Priority	Description
Formal LangGraph StateGraph	High	Replace imperative chat() with proper StateGraph, enabling parallelism, interrupts, and visual debugging
Multi-document support	High	Persistent ChromaDB with namespace isolation per document; metadata filtering at retrieval
Real HITL handoff	High	Webhook to ticketing system (Zendesk API); session ID and conversation history attached
Persistent memory	Medium	LangChain ConversationBufferMemory or Redis-backed memory for multi-turn context
Streaming responses	Medium	Generator-based Phi-2 output for token-by-token streaming in web UI
Evaluation pipeline	Medium	RAGAS integration; automated quality regression testing on each document update
Feedback loops	Medium	Thumbs up/down UI; incorrect answers fed back to improve retrieval thresholds
Model upgrade path	Low	Replace Phi-2 with a larger instruction-tuned model (Llama-3, Mistral) when resources allow
Quantization	Low	4-bit quantization via bitsandbytes to reduce GPU memory and increase throughput
REST API deployment	Low	FastAPI wrapper around chat() with async handling; Docker containerization

— End of Technical Documentation —